

Optimization

2.5 Newton–Raphson method

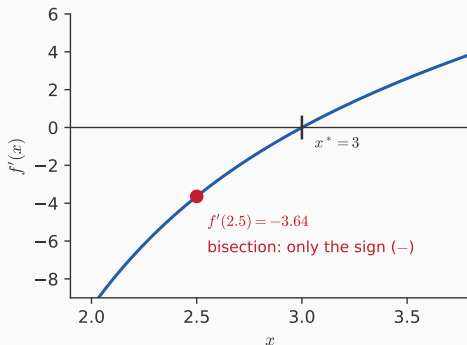
José Firmino Aguilar Madeira

Bisection knows which side. Can we estimate where?

In 2.4, at $x_0 = 2.5$: $f'(2.5) = -3.64$. Bisection only reads the **sign** and concludes $x^* > 2.5$.

But we have the *value* of f' and, if we have f'' , its *slope*: $f''(x) = 2 + 108/x^3$, $f''(2.5) = 8.912$.

Where does the tangent to f' predict that $f' = 0$?



Bisection knows which side. Can we estimate where?

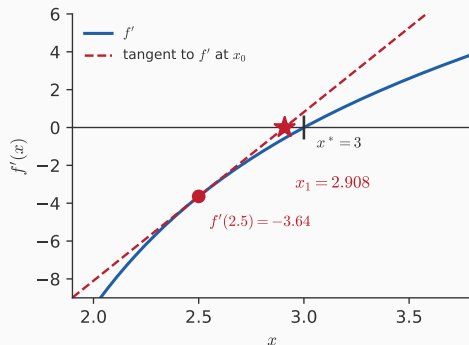
In 2.4, at $x_0 = 2.5$: $f'(2.5) = -3.64$. Bisection only reads the **sign** and concludes $x^* > 2.5$.

But we have the *value* of f' and, if we have f'' , its *slope*: $f''(x) = 2 + 108/x^3$, $f''(2.5) = 8.912$.

Where does the tangent to f' predict that $f' = 0$?

$f'(x) \approx f'(x_0) + f''(x_0)(x - x_0) = 0$:

$$\boxed{x_1 = x_0 - \frac{f'(x_0)}{f''(x_0)}} = 2.5 + \frac{3.64}{8.912} = 2.908.$$



Bisection knows which side. Can we estimate where?

In 2.4, at $x_0 = 2.5$: $f'(2.5) = -3.64$. Bisection only reads the **sign** and concludes $x^* > 2.5$.

But we have the *value* of f' and, if we have f'' , its *slope*: $f''(x) = 2 + 108/x^3$, $f''(2.5) = 8.912$.

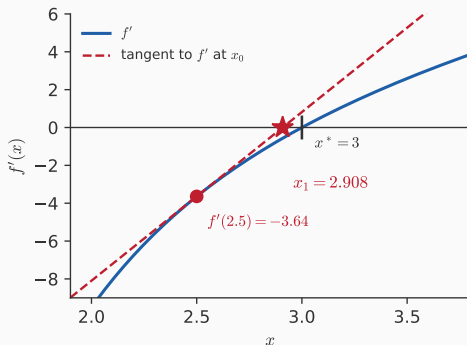
Where does the tangent to f' predict that $f' = 0$?

$f'(x) \approx f'(x_0) + f''(x_0)(x - x_0) = 0$:

$$x_1 = x_0 - \frac{f'(x_0)}{f''(x_0)} = 2.5 + \frac{3.64}{8.912} = 2.908.$$

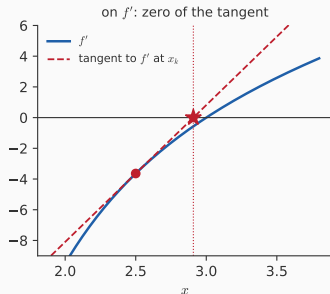
Bisection at 2.5: “the minimum is to the right”.

Newton at 2.5: “it is close to 2.91” — and $x^* = 3$.

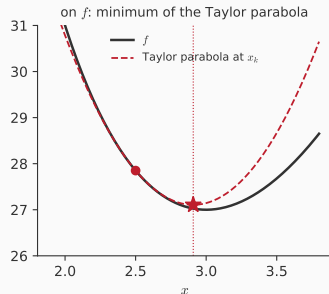


bisection: the **sign** of f' $\rightarrow$ Newton–Raphson: the **value** of f' and its **slope** f''

Two views of the same step



On f' : zero of the tangent,
 $f'(x_k) + f''(x_k)(x - x_k) = 0$.



On f : minimum of the Taylor parabola
 $q(x) = f(x_k) + f'(x_k)(x - x_k) + \frac{1}{2}f''(x_k)(x - x_k)^2$;
 $q'(x) = 0$ is *the same equation*.

Both give $x_{k+1} = x_k - f'(x_k)/f''(x_k)$. As in successive quadratic interpolation (2.3), a parabolic **model** of f — but built from **1 point and 2 derivatives**, not 3 points. q has a minimum only if $f''(x_k) > 0$.

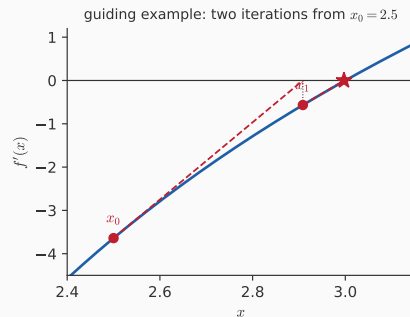
One iteration by hand, and the following ones

$$f(x) = x^2 + 54/x, \quad x_0 = 2.5: \quad f'(x) = 2x - 54/x^2, \\ f''(x) = 2 + 108/x^3.$$

k	x_k	$f'(x_k)$	$f''(x_k)$	x_{k+1}	$ x_{k+1} - 3 $
0	2.5000000	-3.64000	8.9120	2.9084381	$9.2 \cdot 10^{-2}$
1	2.9084381	-0.56685	6.3898	2.9971495	$2.9 \cdot 10^{-3}$
2	2.9971495	-0.01712	6.0114	2.9999973	$2.7 \cdot 10^{-6}$
3	2.9999973	-0.00002	6.0000	3.0000000	$2.4 \cdot 10^{-12}$

Errors $9.2 \cdot 10^{-2} \rightarrow 2.9 \cdot 10^{-3} \rightarrow 2.7 \cdot 10^{-6} \rightarrow 2.4 \cdot 10^{-12}$ in 4 iterations; the 5th reaches **machine precision**. Bisection on $[1, 4]$, with 4 reductions ($n_g = 6$, counting $f'(1)$ and $f'(4)$), only guarantees $|x - x^*| \leq 3/32 \approx 0.09$.

But each Newton iteration requires f' and f'' .



The algorithm

Newton–Raphson: $x_0, \varepsilon_g, k_{\max}$

```
1: set  $k = 0$ ; compute  $f'(x_0)$ 
2: loop
3:   compute  $f''(x_k)$ 
4:   set  $x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}$ 
5:   compute  $f'(x_{k+1})$ 
6:   if  $|f'(x_{k+1})| < \varepsilon_g$  or  $k + 1 \geq k_{\max}$  then
     return  $x_{k+1}$ 
7:   end if
8:   set  $k = k + 1$ 
9: end loop
```

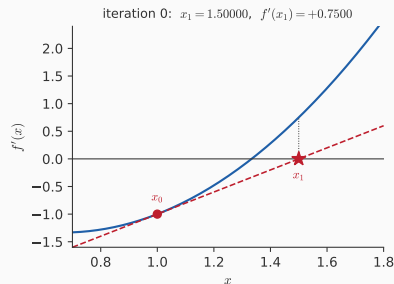
- **Order 2** information: f' and f'' at each point; it does not use f . No interval — only an initial point.
- Cost per iteration: $n_g = 1$, $n_H = 1$ (one evaluation of f' and one of f''); $n_f = 0$.
- Stopping on $|f'|$ (residual of the equation); $|x_{k+1} - x_k| \leq \varepsilon_x$ may be added.
- **Local quadratic** convergence near x^* , under regularity conditions and with $f''(x^*) \neq 0$: the correct digits *tend to double* per iteration.

Newton looks for **zeros of f'** , not minima.

Why do we say “quadratic convergence”?

$$f(x) = x^3 - 2x^2 + 4, \quad x_0 = 1, \quad \varepsilon_g = 10^{-3}: \quad f'(x) = 3x^2 - 4x, \quad f''(x) = 6x - 4.$$

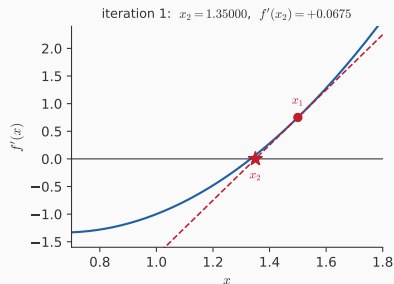
k	x_k	$f'(x_k)$	$f''(x_k)$	x_{k+1}	$f'(x_{k+1})$
0	1.00000	-1.00000	2.0000	1.50000	+0.75000



Why do we say “quadratic convergence”?

$$f(x) = x^3 - 2x^2 + 4, \quad x_0 = 1, \quad \varepsilon_g = 10^{-3}: \quad f'(x) = 3x^2 - 4x, \quad f''(x) = 6x - 4.$$

k	x_k	$f'(x_k)$	$f''(x_k)$	x_{k+1}	$f'(x_{k+1})$
0	1.00000	-1.00000	2.0000	1.50000	+0.75000
1	1.50000	+0.75000	5.0000	1.35000	+0.06750



Why do we say “quadratic convergence”?

$$f(x) = x^3 - 2x^2 + 4, \quad x_0 = 1, \quad \varepsilon_g = 10^{-3}: \quad f'(x) = 3x^2 - 4x, \quad f''(x) = 6x - 4.$$

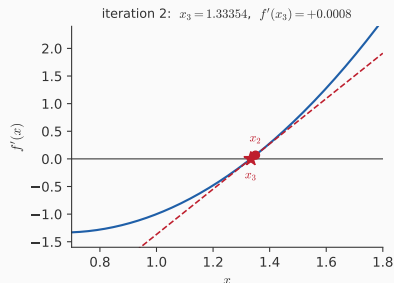
k	x_k	$f'(x_k)$	$f''(x_k)$	x_{k+1}	$f'(x_{k+1})$
0	1.00000	-1.00000	2.0000	1.50000	+0.75000
1	1.50000	+0.75000	5.0000	1.35000	+0.06750
2	1.35000	+0.06750	4.1000	1.33354	+0.00081

$|f'(x_3)| = 8.1 \cdot 10^{-4} < \varepsilon_g$: stop. $x^* = 4/3$ ($f''(4/3) = 4 > 0$, minimum). Errors $e_k = |x_k - x^*|$:

$0.33 \rightarrow 0.17 \rightarrow 0.017 \rightarrow 0.0002$.

Near x^* , $\boxed{e_{k+1} \approx C e_k^2}$, $C = \left| \frac{f'''(x^*)}{2f''(x^*)} \right|$ ($= 0.75$ here; $1/3$ in

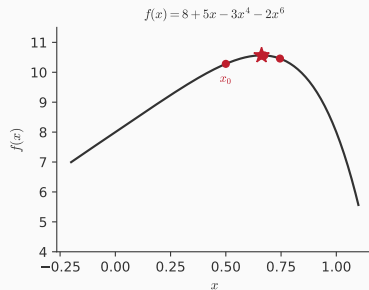
the guiding example, where $e_{k+1}/e_k^2 \rightarrow 0.333$). If $e_k \approx 10^{-p}$, $e_{k+1} \approx C 10^{-2p}$: the number of correct digits **tends to double** (2.1).



Newton looks for $f' = 0$, not minima

Maximize $f(x) = 8 + 5x - 3x^4 - 2x^6$, $x_0 = 0.5$, $\varepsilon_g = 10^{-7}$: $f'(x) = 5 - 12(x^3 + x^5)$,
 $f''(x) = -12(3x^2 + 5x^4)$. The formula **does not know** whether we want a minimum or a maximum: it only solves $f'(x) = 0$. Here $f'' < 0$ and the point is a maximum.

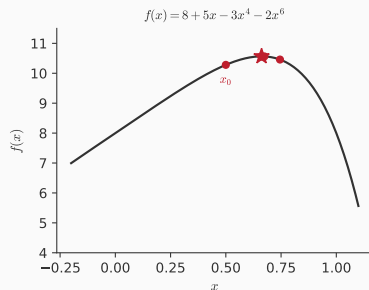
k	x_k	$f(x_k)$	$f'(x_k)$	$f''(x_k)$	x_{k+1}
0	0.5000000	10.281250	+3.125000	-12.75000	0.7450980



Newton looks for $f' = 0$, not minima

Maximize $f(x) = 8 + 5x - 3x^4 - 2x^6$, $x_0 = 0.5$, $\varepsilon_g = 10^{-7}$: $f'(x) = 5 - 12(x^3 + x^5)$,
 $f''(x) = -12(3x^2 + 5x^4)$. The formula **does not know** whether we want a minimum or a maximum: it only solves $f'(x) = 0$. Here $f'' < 0$ and the point is a maximum.

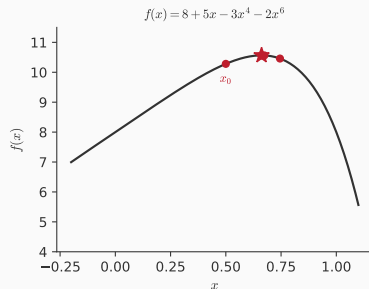
k	x_k	$f(x_k)$	$f'(x_k)$	$f''(x_k)$	x_{k+1}
0	0.5000000	10.281250	+3.125000	-12.75000	0.7450980
1	0.7450980	10.458621	-2.719687	-38.47906	0.6744184



Newton looks for $f' = 0$, not minima

Maximize $f(x) = 8 + 5x - 3x^4 - 2x^6$, $x_0 = 0.5$, $\varepsilon_g = 10^{-7}$: $f'(x) = 5 - 12(x^3 + x^5)$,
 $f''(x) = -12(3x^2 + 5x^4)$. The formula **does not know** whether we want a minimum or a maximum: it only solves $f'(x) = 0$. Here $f'' < 0$ and the point is a maximum.

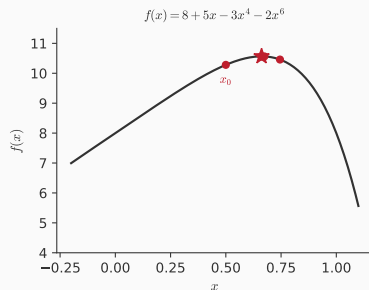
k	x_k	$f(x_k)$	$f'(x_k)$	$f''(x_k)$	x_{k+1}
0	0.5000000	10.281250	+3.125000	-12.75000	0.7450980
1	0.7450980	10.458621	-2.719687	-38.47906	0.6744184
2	0.6744184	10.563259	-0.355311	-28.78702	0.6620756



Newton looks for $f' = 0$, not minima

Maximize $f(x) = 8 + 5x - 3x^4 - 2x^6$, $x_0 = 0.5$, $\varepsilon_g = 10^{-7}$: $f'(x) = 5 - 12(x^3 + x^5)$,
 $f''(x) = -12(3x^2 + 5x^4)$. The formula **does not know** whether we want a minimum or a maximum: it only solves $f'(x) = 0$. Here $f'' < 0$ and the point is a maximum.

k	x_k	$f(x_k)$	$f'(x_k)$	$f''(x_k)$	x_{k+1}
0	0.5000000	10.281250	+3.125000	-12.75000	0.7450980
1	0.7450980	10.458621	-2.719687	-38.47906	0.6744184
2	0.6744184	10.563259	-0.355311	-28.78702	0.6620756
3	0.6620756	10.565490	-0.009182	-27.30912	0.6617394



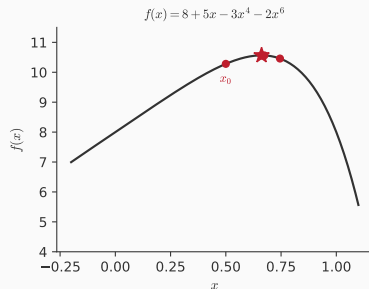
Newton looks for $f' = 0$, not minima

Maximize $f(x) = 8 + 5x - 3x^4 - 2x^6$, $x_0 = 0.5$, $\varepsilon_g = 10^{-7}$: $f'(x) = 5 - 12(x^3 + x^5)$,
 $f''(x) = -12(3x^2 + 5x^4)$. The formula **does not know** whether we want a minimum or a maximum: it only solves $f'(x) = 0$. Here $f'' < 0$ and the point is a maximum.

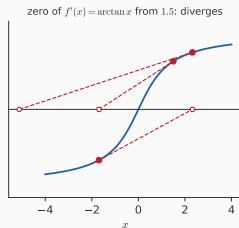
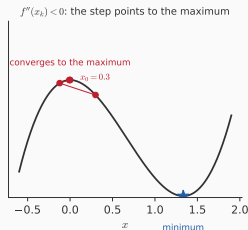
k	x_k	$f(x_k)$	$f'(x_k)$	$f''(x_k)$	x_{k+1}
0	0.5000000	10.281250	+3.125000	-12.75000	0.7450980
1	0.7450980	10.458621	-2.719687	-38.47906	0.6744184
2	0.6744184	10.563259	-0.355311	-28.78702	0.6620756
3	0.6620756	10.565490	-0.009182	-27.30912	0.6617394
4	0.6617394	10.565491	-0.000007	-27.26970	0.6617391

After 5 iterations $|f'| < 10^{-7}$: $x^* = 0.6617391$,
 $f(x^*) = 10.565491$ (correct digits 1, 2, 3, 6: they tend to double).

Classify afterwards: $f''(x^*) > 0$ minimum; $f''(x^*) < 0$ maximum;
 $f''(x^*) = 0$ inconclusive.



When it fails, and possible safeguards



- **Wrong stationary point:** it may converge to another zero of f' (minimum, maximum or degenerate stationary point) depending on x_0 . In the cubic $x^3 - 2x^2 + 4$, from $x_0 = 0.3$: maximum at $x = 0$.
- $f''(x_k) \approx 0$: the step $-f'/f''$ becomes huge.
- **Initial point far away:** quadratic convergence is *local*; the method may oscillate or diverge (on the right, Newton for $\arctan x = 0$ from 1.5).

Possible safeguards: locate first, refine afterwards (e.g. bisection $\rightarrow$ Newton); do not accept a step that leaves the known interval or does not improve f — reduce/damp the step (Ch. 3).

Safeguarded Newton: bisection + Newton

An interval $[a, b]$ with $f'(a) < 0 < f'(b)$ is maintained. At each iteration: (1) try the Newton step; (2) if it leaves (a, b) or does not make enough progress, take a bisection step; (3) update the interval with the sign of f' at the new point. Near the solution, Newton prevails.

robustness of bisection
+ speed of Newton

Same philosophy as Brent's method: a fast step with a robust safeguard (guarantees bracketing).

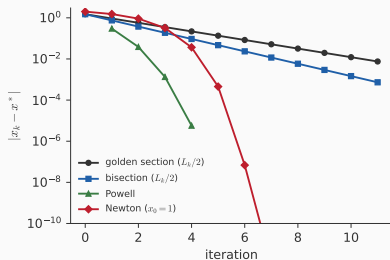
derivative-free: golden section + interpolation

with derivatives: bisection + Newton

Example: $f'(x) = \arctan x$ (minimum of f at 0), $[-1, 2]$, $x_0 = 1.5$ — the case that diverged (the initial interval is given; only the new points update it). One bisection step, then three Newton steps:

k	x_k	Newton point	x_{k+1}	new interval
0	1.5	-1.694 : outside $(a, b) \Rightarrow$ bisection	0.5	$[-1, 0.5]$
1	0.5	-0.0796 : accepted	-0.0796	$[-0.0796, 0.5]$
2	-0.0796	$3.4 \cdot 10^{-4}$: accepted	$3.4 \cdot 10^{-4}$	$[-0.0796, 3.4 \cdot 10^{-4}]$
3	$3.4 \cdot 10^{-4}$	$-2.5 \cdot 10^{-11}$: accepted	$-2.5 \cdot 10^{-11}$	

The four methods on $f(x) = x^2 + 54/x$



On this problem, with these parameters ($[1, 4]$; Powell $x_1 = 1$, $\Delta = 1$; Newton $x_0 = 1$):

Method	Deriv. (order)	Error after 5 it.	Cost
Golden section	0	$L_5/2 = 0.14$	$n_f = 7$
Bisection	1	$L_5/2 = 0.047$	$n_g = 7$
Powell	0	$6 \cdot 10^{-6}$ (4 it.)	$n_f = 7$
Newton	2	$4.5 \cdot 10^{-4}$ (7th: 10^{-15})	$n_g = 6, n_H = 5$

Interval: predictable and slow error. **Estimation:** may be irregular far from the solution, but very fast near it. Newton starts from $x_0 = 1$, far away: 4 iterations to “get close”, then 3 give 10^{-15} (from $x_0 = 2.5$: $2.4 \cdot 10^{-12}$ in 4; the 5th reaches machine precision).

Per iteration: golden section and quadratic interpolation, 1 evaluation of f (plus 2 and 3 initial ones); bisection, 1 of f' (plus $f'(a)$, $f'(b)$); Newton, 1 of f' and 1 of f'' (plus $f'(x_0)$) — or 3 of f , $f(x \pm h)$ and $f(x)$, *reused* for both derivatives (2.6).

The core, in MATLAB:

```
g = df(x);  
for k = 1:kmax  
    H = ddf(x);  
    if abs(H) < tolH  
        error('curvature too small')  
    end  
    x = x - g/H;           % Newton step  
    g = df(x);             % 1 f' evaluation  
    if abs(g) < tolg, break, end  
end
```

The full function (Newton1D, with counts and a warning if the iteration limit is reached) is in the course companion code.

Per iteration, with analytical derivatives:

$$n_g = 1, n_H = 1, n_f = 0.$$

Without analytical derivatives $\rightarrow$ 2.6: $f(x-h)$, $f(x)$, $f(x+h)$ give f' and f'' at the same time

— 3 evaluations of f per point, if reused.

At the end, **classify**: $f''(x) > 0$ minimum;
 $f''(x) < 0$ maximum.

KEY TAKEAWAYS — 2.5

1. Bisection uses the *sign* of f' ; Newton uses its *value* and its slope f'' to **estimate where** the zero is: $x_{k+1} = x_k - f'(x_k)/f''(x_k)$.
2. Two views of the same step: zero of the tangent to $f' =$ minimum of the Taylor parabola of f (if $f''(x_k) > 0$).
3. **Local quadratic** convergence ($e_{k+1} \approx Ce_k^2$, $f''(x^*) \neq 0$, under regularity): the correct digits tend to double.
4. Newton looks for **zeros of f'** , not minima: it may go to maxima, diverge or take huge steps if $f'' \approx 0$. Safeguard: locate first, refine afterwards (bisection + Newton).

Next (2.6): Newton is very fast when we have f' and f'' . **What if the code only returns $f(x)$?**
Can we build f' and f'' , approximately, from evaluations of f ? — finite differences.

References Full list: *Course bibliography*, at the end of the course PDF.

- Deb, K., *Optimization for Engineering Design: Algorithms and Examples*, 2nd ed., PHI Learning, 2012 — Ch. 2.
- Dennis, J. E., Jr., Schnabel, R. B., *Numerical Methods for Unconstrained Optimization and Nonlinear Equations*, SIAM (Classics in Applied Mathematics 16), 1996 — Ch. 2.

Resources

- Companion code: `Newton1D` (counts, warning if the maximum number of iterations is reached).